ASP.NET CORE ESSENTIALS

Module 16-20

Table of Contents

Module 16: Dependency Inversion Using DataAccessFactory ... 3

16.1 Understanding the Problem: Tight Coupling ... 3

16.2 Introducing DataAccessFactory ... 3

16.2.1 Repository Interfaces in DAL ... 4

16.2.2 Concrete Repository Implementations with Internal Access Modifier ... 4

16.2.3 Implementing the DataAccessFactory ... 5

16.4 Integrating with Dependency Injection in API ... 6

16.5 Summary ... 6

Module 17: DTOs and AutoMapper in 3-Tier Architecture ... 7

17.1 What is a DTO? ... 7

17.2 Why AutoMapper? ... 7

17.3 Setting Up DTOs in 3-Tier Architecture ... 8

17.4 Configuring AutoMapper ... 8

17.4.1 Register AutoMapper in API ... 9

17.5 Using DTOs in BLL ... 9

17.6 Using DTOs in API Controllers ... 10

17.7 Summary ... 10

Module 18: API Authentication (Token-Based) ... 11

18.1 How Token-Based Authentication Works ... 11

18.2 3-Tier Implementation ... 11

18.2.1 BLL – Token Generation ... 11

18.2.2 API – Custom Attribute for Token Validation ... 12

18.2.3 Using the Attribute in API Controller ... 13

18.2.4 Dependency Injection in API ... 13

18.3 Summary ... 14

Module 19: HttpOnly Cookie-Based Authentication using IAsyncAuthorizationFilter ... 15

19.1 Why Use IAsyncAuthorizationFilter ... 15

19.2 AuthService in BLL ... 15

19.3 Custom IAsyncAuthorizationFilter Attribute ... 16

19.4 Using the Attribute in API Controllers ... 17

19.5 API Login/Logout with HttpOnly Cookie ... 17

19.6 Dependency Injection Setup ... 18

19.7 Benefits of Async Authorization Filter ... 19

19.8 Summary 19
Module 20: Secure API Authentication for Any Client 20
20.1 Goals of Secure API Authentication 20
20.2 Authentication Flow for Any Client 20
20.3 AuthService in BLL 21
20.4 Custom IAsyncAuthorizationFilter for Multi-Client 22
20.5 API Controller Example 22
20.6 Login Endpoint for Any Client 23
20.7 Dependency Injection Setup 24
20.8 Security Best Practices 24
20.9 Summary 25

Module 16: Dependency Inversion Using DataAccessFactory

Modern software development follows the **SOLID principles**, and in particular, the **Dependency Inversion Principle (DIP)**. When designing a **3-tier architecture**, it is common to see **tight coupling between BLL and DAL** if we directly instantiate repository classes. This module introduces the **DataAccessFactory** pattern to decouple layers, improve maintainability, and make unit testing easier.

16.1 Understanding the Problem: Tight Coupling

In a naive implementation, the BLL might create concrete repository instances like this:

```
public class StudentService
{
    private readonly StudentRepository _repo = new StudentRepository(); // tightly coupled

    public void CreateStudent(Student student)
    {
        _repo.Add(student);
    }
}
```

Why this is a problem:

1. **Violation of DIP:** BLL depends on a concrete DAL class instead of an abstraction (interface).
2. **Hard to unit test:** Cannot mock `StudentRepository` easily.
3. **Maintenance issues:** Any change in DAL class requires changes in BLL.
4. **Limited flexibility:** Adding a new repository type requires modifying BLL code.

To solve this, we introduce **DataAccessFactory**, which returns repository **interfaces** instead of concrete classes.

16.2 Introducing DataAccessFactory

DataAccessFactory is a centralized class in **DAL** responsible for creating objects that implement repository interfaces.

- The **factory returns interfaces** for all repository types.
- The **BLL only depends on interfaces**, not concrete implementations.
- Multiple repositories can be accessed through **separate methods** in the factory.

16.2.1 Repository Interfaces in DAL

```
public interface IStudentRepository
{
    void Add(Student student);
    Student Get(int id);
    IEnumerable<Student> GetAll();
}

public interface ICourseRepository
{
    void Add(Course course);
    Course Get(int id);
    IEnumerable<Course> GetAll();
}
```

16.2.2 Concrete Repository Implementations with Internal Access Modifier

```
internal class StudentRepository : IStudentRepository
{
    private readonly SchoolDbContext _context;
    public StudentRepository(SchoolDbContext context) => _context = context;

    public void Add(Student student) => _context.Students.Add(student);
    public Student Get(int id) => _context.Students.Find(id);
    public IEnumerable<Student> GetAll() => _context.Students.ToList();
}

internal class CourseRepository : ICourseRepository
{
    private readonly SchoolDbContext _context;
    public CourseRepository(SchoolDbContext context) => _context = context;

    public void Add(Course course) => _context.Courses.Add(course);
    public Course Get(int id) => _context.Courses.Find(id);
    public IEnumerable<Course> GetAll() => _context.Courses.ToList();
}
```

16.2.3 Implementing the DataAccessFactory

```
public class DataAccessFactory
{
    private readonly SchoolDbContext _context;

    public DataAccessFactory(SchoolDbContext context)
    {
        _context = context;
    }

    public IStudentRepository GetStudentRepository()
    {
        return new StudentRepository(_context);
    }

    public ICourseRepository GetCourseRepository()
    {
        return new CourseRepository(_context);
    }

    // Add more methods for additional repositories as needed
}
```

Each method returns an **interface**, hiding the concrete implementation. This ensures **BLL depends only on abstractions**.

16.3 Injecting DataAccessFactory into BLL

Instead of injecting individual repositories, we inject **DataAccessFactory** into the service class:

```
public class StudentService
{
    private readonly IStudentRepository _studentRepo;
    public StudentService(DataAccessFactory factory)
    {
        _studentRepo = factory.GetStudentRepository();
    }
    public void CreateStudent(Student student)
    {
        if (string.IsNullOrEmpty(student.Name))
            throw new Exception("Student name is required");

        _studentRepo.Add(student);
    }
    public IEnumerable<Student> GetAllStudents()
    {
        return _studentRepo.GetAll();
    }
}
```

Benefits of this approach:

- The **service layer is decoupled from DAL implementations**.
- Adding a new repository (like `ICourseRepository`) doesn't require modifying services that don't use it.
- Supports **unit testing**: you can inject a **mock factory** that returns mocked repositories.

16.4 Integrating with Dependency Injection in API

In `Program.cs` of the API project:

```
builder.Services.AddScoped<DataAccessFactory>();
builder.Services.AddScoped<StudentService>();
```

The **factory is registered** once in DI, and all BLL services can use it to obtain repository interfaces.

16.5 Summary

1. **Problem:** Direct instantiation of DAL repositories in BLL causes **tight coupling**, poor testability, and maintenance issues.
2. **Solution:** Introduce a **DataAccessFactory** in DAL that returns repository interfaces.
3. **Implementation:**
    - Define **repository interfaces** in DAL
    - Implement **concrete repositories**
    - Create **DataAccessFactory** with methods returning interface objects
    - Inject **factory into BLL services**
4. **Benefits:**
    - BLL depends only on **abstractions**
    - Flexible, maintainable, and testable
    - Supports adding new repositories without modifying BLL
    - Clean **3-tier architecture adherence**

By using **DataAccessFactory**, your 3-tier solution becomes **loosely coupled, SOLID-compliant, and ready for extensions** like generic repositories or DTO mapping.

Module 17: DTOs and AutoMapper in 3-Tier Architecture

When building modern web applications, especially with **APIs**, it is not recommended to expose your **entity models directly** to clients. This can lead to:

- **Overexposing sensitive data**
- Tight coupling between API and database schema
- Difficulties in maintaining backward compatibility

To solve this, we use **DTOs (Data Transfer Objects)** along with **AutoMapper** for **clean, maintainable, and secure data transfer** between layers.

17.1 What is a DTO?

A **DTO (Data Transfer Object)** is a **plain object** that carries data between layers or across networks.

Example:

```
public class StudentDTO
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Benefits:

- Only exposes **necessary fields** to the client
- **Decouples** API contracts from database entities
- Facilitates **versioning** and future changes

17.2 Why AutoMapper?

AutoMapper is a library that **automatically maps properties** from one object to another:

- Maps **Entity → DTO** or **DTO → Entity**

- Eliminates manual copying of properties
- Reduces boilerplate code in BLL

Example:

```
var studentDto = _mapper.Map<StudentDTO>(studentEntity);
```

AutoMapper automatically maps properties with **matching names**.

17.3 Setting Up DTOs in 3-Tier Architecture

Step 1: Create a Common Project for DTOs

```
MyApp.Common/
 └─ DTOs/
     └─ StudentDTO.cs
```

- DTOs live in **Common project** to be accessible by **BLL and API**
- Entities remain in **DAL**, keeping separation intact

Step 2: Install AutoMapper Packages

In **API project** (startup project):

```
dotnet add package AutoMapper
dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
```

17.4 Configuring AutoMapper

Create a **mapping profile** in BLL or Common project:

```
using AutoMapper;
using MyApp.DAL;
using MyApp.Common.DTOs;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Student, StudentDTO>().ReverseMap();
        // Add more entity ↔ DTO mappings here
    }
}
```

Explanation:

- `CreateMap<Student, StudentDTO>()` maps entity to DTO
- `.ReverseMap()` enables DTO → Entity mapping (useful for POST/PUT operations)

17.4.1 Register AutoMapper in API

In `Program.cs`:

```
builder.Services.AddAutoMapper(typeof(MappingProfile));
```

Now AutoMapper can be **injected** into BLL services via DI.

17.5 Using DTOs in BLL

```
public class StudentService
{
    private readonly IStudentRepository _repo;
    private readonly IMapper _mapper;

    public StudentService(DataAccessFactory factory, IMapper mapper)
    {
        _repo = factory.GetStudentRepository();
        _mapper = mapper;
    }

    public IEnumerable<StudentDTO> GetAllStudents()
    {
        var students = _repo.GetAll();
        return _mapper.Map<IEnumerable<StudentDTO>>(students);
    }

    public void CreateStudent(StudentDTO dto)
    {
        var student = _mapper.Map<Student>(dto);
        _repo.Add(student);
    }
}
```

Key Points:

- **BLL never exposes entities directly**
- **DTOs flow from BLL to API** and back
- AutoMapper handles **conversion automatically**

17.6 Using DTOs in API Controllers

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly StudentService _studentService;

    public StudentsController(StudentService studentService)
    {
        _studentService = studentService;
    }

    [HttpGet]
    public IActionResult Get()
    {
        var students = _studentService.GetAllStudents();
        return Ok(students);
    }

    [HttpPost]
    public IActionResult Create(StudentDTO dto)
    {
        _studentService.CreateStudent(dto);
        return Ok();
    }
}
```

The API **only deals with DTOs**, keeping database entities hidden.

17.7 Summary

- **DTOs**: Transfer data safely between layers, decoupling API from DAL.
- **AutoMapper**: Automatically maps entities to DTOs and vice versa.
- **3-Tier Flow**:
    1. API receives DTO → passes to BLL
    2. BLL converts DTO → Entity using AutoMapper
    3. DAL saves entity to database
    4. Retrieval: DAL → Entity → BLL → DTO → API → Client
- **Benefits:** Cleaner code, better security, maintainability, and easier testing.

With **DTOs and AutoMapper** in place, your 3-tier architecture is **fully decoupled** and ready for **robust API development**.

Module 18: API Authentication (Token-Based)

Authentication is a critical part of any API. In a **3-tier architecture**, we want to **secure our endpoints** without tightly coupling the API to the database or business logic.

A **token-based approach** achieves this by issuing a **token** on login that the client sends with each request.

18.1 How Token-Based Authentication Works

Flow Overview:

1. Client sends **login credentials** (username/password) to API.
2. API forwards credentials to **BLL**, which validates the user using DAL.
3. If valid, BLL generates a **token** (JWT or custom) and stores it in **database or cache**.
4. API returns the token to the client.
5. Client includes the token in the **Authorization header** in subsequent requests.
6. API validates the token using a **custom attribute** before granting access.

This keeps authentication logic **centralized in BLL** while API focuses on **request handling and validation**.

18.2 3-Tier Implementation

18.2.1 BLL – Token Generation

```
public class AuthService
{
    private readonly IUserRepository _userRepo;

    public AuthService(DataAccessFactory factory)
    {
        _userRepo = factory.GetUserRepository();
    }

    public string Login(string username, string password)
    {
        var user = _userRepo.GetByUsername(username);
```

```
        if (user == null || user.Password != password)
            throw new Exception("Invalid credentials");

        // Create token (simple GUID for demonstration)
        var token = Guid.NewGuid().ToString();

        // Store token in DB for validation
        user.Token = token;
        _userRepo.Update(user);

        return token;
    }

    public bool ValidateToken(string token)
    {
        return _userRepo.GetByToken(token) != null;
    }
}
```

Key Points:

- BLL validates credentials and issues token
- Token is stored in **database** for tracking and revocation
- **BLL never depends on API**, keeping layers decoupled

18.2.2 API – Custom Attribute for Token Validation

Create a **custom authorization attribute**:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

public class TokenAuthAttribute : ActionFilterAttribute
{
    private readonly AuthService _authService;

    public TokenAuthAttribute(AuthService authService)
    {
        _authService = authService;
    }

    public override void OnActionExecuting(ActionExecutingContext context)
    {
        var token = context.HttpContext.Request.Headers["Authorization"].FirstOrDefault();
        if (string.IsNullOrEmpty(token) || !_authService.ValidateToken(token))
        {
            context.Result = new UnauthorizedResult();
            return;
        }
```

```
        base.OnActionExecuting(context);
    }
}
```

Explanation:

- This **filter checks the token** before the action executes
- If invalid, **returns 401 Unauthorized**
- Can be applied **per controller or action**

18.2.3 Using the Attribute in API Controller

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly StudentService _studentService;

    public StudentsController(StudentService studentService)
    {
        _studentService = studentService;
    }

    [HttpGet]
    [ServiceFilter(typeof(TokenAuthAttribute))]
    public IActionResult GetStudents()
    {
        var students = _studentService.GetAllStudents();
        return Ok(students);
    }
}
```

Only requests with **valid token** can access this endpoint.

18.2.4 Dependency Injection in API

In `Program.cs`:

```
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<TokenAuthAttribute>();
builder.Services.AddScoped<DataAccessFactory>();
builder.Services.AddScoped<StudentService>();
```

- AuthService depends on **DataAccessFactory** to access **user repository**
- TokenAuthAttribute depends on **AuthService**
- API controller depends on **services** only

18.3 Summary

- Token-based authentication allows **secure API access** without tightly coupling API to DAL.
- **BLL handles token generation and validation**, ensuring business logic is centralized.
- **Custom attribute in API** checks the token on each request.
- Using **DataAccessFactory**, API and BLL remain decoupled, respecting **SOLID principles**.
- Easily extendable to **JWT tokens** for stateless authentication.

By implementing token-based authentication, your 3-tier API becomes **secure, maintainable, and testable**.

Module 19: HttpOnly Cookie-Based Authentication using IAsyncAuthorizationFilter

---

When building APIs, it's often necessary to **perform authentication checks asynchronously**, for example, when querying the database to validate a token. ASP.NET Core provides **`IAuthorizationFilter`** and **`IAsyncAuthorizationFilter`** interfaces for this purpose.

Using `IAsyncAuthorizationFilter` allows us to **validate HttpOnly cookie tokens asynchronously** while keeping **3-tier architecture principles** intact.

19.1 Why Use IAsyncAuthorizationFilter

- Async execution allows **database queries or external service calls** without blocking threads.
- **Centralizes authorization logic** per controller or per action.
- Keeps API **thin**, delegating token validation to **BLL**.
- Fully compatible with **HttpOnly cookies** for web or SPA clients.

19.2 AuthService in BLL

BLL handles token creation, validation, and logout. Using the same **DataAccessFactory** pattern:

```
public class AuthService
{
    private readonly IUserRepository _userRepo;

    public AuthService(DataAccessFactory factory)
    {
        _userRepo = factory.GetUserRepository();
    }

    public string Login(string username, string password)
    {
        var user = _userRepo.GetByUsername(username);
        if (user == null || user.Password != password)
            throw new Exception("Invalid credentials");
```

```
        var token = Guid.NewGuid().ToString();
        user.Token = token;
        _userRepo.Update(user);

        return token;
    }

    public async Task<bool> ValidateToken (string token)
    {
        var user = await _userRepo.GetByToken (token);
        return user != null;
    }

    public async Task LogoutAsync(string token)
    {
        var user = await _userRepo.GetByTokenAsync(token);
        if (user != null)
        {
            user.Token = null;
            await _userRepo.UpdateAsync(user);
        }
    }
}
```

Note: All token validation operations are **async**, compatible with `IAsyncAuthorizationFilter`.

19.3 Custom IAsyncAuthorizationFilter Attribute

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

public class CookieTokenAuthAsyncAttribute : Attribute, IAsyncAuthorizationFilter
{
    private readonly AuthService _authService;

    public CookieTokenAuthAsyncAttribute(AuthService authService)
    {
        _authService = authService;
    }

    public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
    {
        var request = context.HttpContext.Request;

        if (!request.Cookies.TryGetValue("AuthToken", out var token) ||
            !await _authService.ValidateTokenAsync(token))
        {
            context.Result = new UnauthorizedResult();
            return;
        }
    }
```

```
}
```

Key Points:

- Implements `IAsyncAuthorizationFilter` → supports async DB calls
- Checks **HttpOnly cookie** named `AuthToken`
- Returns `401 Unauthorized` if token is invalid or missing

19.4 Using the Attribute in API Controllers

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly StudentService _studentService;

    public StudentsController(StudentService studentService)
    {
        _studentService = studentService;
    }

    [HttpGet]
    [ServiceFilter(typeof(CookieTokenAuthAsyncAttribute))]
    public IActionResult GetStudents()
    {
        var students = _studentService.GetAllStudents();
        return Ok(students);
    }
}
```

The API automatically enforces **token validation asynchronously** for this endpoint.

19.5 API Login/Logout with HttpOnly Cookie

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("login")]
```

```
    public async Task<IActionResult> Login([FromBody] LoginRequest request)
    {
        try
        {
            var token = _authService.Login(request.Username, request.Password);

            Response.Cookies.Append("AuthToken", token, new CookieOptions
            {
                HttpOnly = true,
                Secure = true,
                SameSite = SameSiteMode.Strict,
                Expires = DateTimeOffset.UtcNow.AddHours(1)
            });

            return Ok(new { Message = "Login successful" });
        }
        catch
        {
            return Unauthorized(new { Message = "Invalid credentials" });
        }
    }

    [HttpPost("logout")]
    public async Task<IActionResult> Logout()
    {
        if (Request.Cookies.TryGetValue("AuthToken", out var token))
        {
            await _authService.LogoutAsync(token);
            Response.Cookies.Delete("AuthToken");
        }

        return Ok(new { Message = "Logout successful" });
    }
}
```

Notes:

- Login sets the token in **HttpOnly cookie**
- Logout deletes the cookie and clears the token from DB
- BLL handles all business rules

19.6 Dependency Injection Setup

```
builder.Services.AddScoped<DataAccessFactory>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<CookieTokenAuthAsyncAttribute>();
builder.Services.AddScoped<StudentService>();
```

The async attribute can now be **injected into controllers**, maintaining **3-tier decoupling**.

19.7 Benefits of Async Authorization Filter

1. **Asynchronous token validation** → non-blocking, scalable
2. **Secures API endpoints using HttpOnly cookies**
3. **Keeps business logic in BLL**, API layer remains thin
4. **Supports clean 3-tier architecture** and testable services

19.8 Summary

- **IAsyncAuthorizationFilter** is ideal for **async token validation** in APIs
- AuthService in BLL handles **token generation, validation, and logout**
- **API endpoints** are protected using the custom async attribute
- **HttpOnly cookies** enhance security for web clients or SPA applications
- **3-tier architecture remains decoupled** using DataAccessFactory and DI

Your API is now **secure, asynchronous, and fully decoupled** from the database while supporting **cookie-based authentication**.

Module 20: Secure API Authentication for Any Client

In modern applications, APIs are consumed by **different types of clients**:

- **Web browsers** (SPA or traditional apps)
- **Mobile apps** (iOS, Android)
- **Embedded devices** (IoT, smart devices)

To secure your API, we need an **authentication mechanism** that works consistently across all clients while **keeping tokens safe** and **preventing common attacks** like XSS, CSRF, or replay attacks.

20.1 Goals of Secure API Authentication

1. **Universal compatibility**: Works for web, mobile, and embedded clients.
2. **Secure token storage**: Tokens must not be easily accessible by attackers.
3. **Separation of concerns**: BLL handles token management; DAL persists tokens; API validates requests.
4. **Scalable and stateless if possible**: Especially for mobile and embedded clients.

We combine **token-based authentication** with **HttpOnly cookies** for web clients, and **Bearer tokens in headers** for mobile/embedded clients.

20.2 Authentication Flow for Any Client

1. **Login request**: Client sends credentials to `/api/auth/login`.
2. **BLL validates user** via DataAccessFactory → IUserRepository.
3. **Token generated** in BLL (JWT or GUID-based).
4. **Token storage**:
    - Web clients: set in **HttpOnly cookie**
    - Mobile/embedded: returned in **response body**, client stores securely.
5. **Protected endpoints**: Custom `IAsyncAuthorizationFilter` validates token (from cookie or header).
6. **Logout**: Invalidates token in DB; deletes cookie (web) or clears local storage (mobile/embedded).

20.3 AuthService in BLL

```
public class AuthService
{
    private readonly IUserRepository _userRepo;

    public AuthService(DataAccessFactory factory)
    {
        _userRepo = factory.GetUserRepository();
    }

    public string Login(string username, string password)
    {
        var user = _userRepo.GetByUsername(username);
        if (user == null || user.Password != password)
            throw new Exception("Invalid credentials");

        var token = Guid.NewGuid().ToString(); // or JWT
        user.Token = token;
        _userRepo.Update(user);

        return token;
    }

    public async Task<bool> ValidateTokenAsync(string token)
    {
        var user = await _userRepo.GetByTokenAsync(token);
        return user != null;
    }

    public async Task LogoutAsync(string token)
    {
        var user = await _userRepo.GetByTokenAsync(token);
        if (user != null)
        {
            user.Token = null;
            await _userRepo.UpdateAsync(user);
        }
    }
}
```

Notes:

- Token generation is **centralized in BLL**
- Token validation is **asynchronous**, allowing **database or cache checks**
- Works for **any client**: web, mobile, or embedded

20.4 Custom IAsyncAuthorizationFilter for Multi-Client

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;

public class MultiClientAuthAttribute : Attribute, IAsyncAuthorizationFilter
{
    private readonly AuthService _authService;

    public MultiClientAuthAttribute(AuthService authService)
    {
        _authService = authService;
    }

    public async Task OnAuthorizationAsync(AuthorizationFilterContext context)
    {
        var request = context.HttpContext.Request;

        string token = null;

        // 1. Check HttpOnly cookie (web clients)
        if (request.Cookies.TryGetValue("AuthToken", out var cookieToken))
            token = cookieToken;

        // 2. Check Authorization header (mobile/embedded clients)
        if (string.IsNullOrEmpty(token) && request.Headers.TryGetValue("Authorization", out
var header))
        {
            var authHeader = header.FirstOrDefault();
            if (!string.IsNullOrEmpty(authHeader) && authHeader.StartsWith("Bearer "))
                token = authHeader.Substring("Bearer ".Length).Trim();
        }

        if (string.IsNullOrEmpty(token) || !await _authService.ValidateTokenAsync(token))
        {
            context.Result = new UnauthorizedResult();
            return;
        }
    }
}
```

Explanation:

- Supports **web clients** via **HttpOnly cookie**
- Supports **mobile/embedded clients** via **Bearer token**
- **Asynchronous validation** prevents thread blocking

20.5 API Controller Example

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    private readonly StudentService _studentService;

    public StudentsController(StudentService studentService)
    {
        _studentService = studentService;
    }

    [HttpGet]
    [ServiceFilter(typeof(MultiClientAuthAttribute))]
    public IActionResult GetAll()
    {
        var students = _studentService.GetAllStudents();
        return Ok(students);
    }
}
```

Notes:

- All requests require a **valid token**, regardless of client type
- API does not care if token comes from **cookie** or **header**

20.6 Login Endpoint for Any Client

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest request, [FromQuery]
bool webClient = true)
    {
        var token = _authService.Login(request.Username, request.Password);

        if (webClient)
        {
            Response.Cookies.Append("AuthToken", token, new CookieOptions
            {
                HttpOnly = true,
                Secure = true,
```

```
                SameSite = SameSiteMode.Strict,
                Expires = DateTimeOffset.UtcNow.AddHours(1)
            });
            return Ok(new { Message = "Login successful (web)" });
        }

        // Mobile / embedded clients
        return Ok(new { Token = token, Message = "Login successful (non-web)" });
    }

    [HttpPost("logout")]
    public async Task<IActionResult> Logout()
    {
        if (Request.Cookies.TryGetValue("AuthToken", out var token))
        {
            await _authService.LogoutAsync(token);
            Response.Cookies.Delete("AuthToken");
        }

        return Ok(new { Message = "Logout successful" });
    }
}
```

Key Points:

- **Web clients** receive token in **HttpOnly cookie**
- **Mobile/embedded clients** receive token in **JSON response**
- Logout invalidates token in **BLL/DAL**

20.7 Dependency Injection Setup

```
builder.Services.AddScoped<DataAccessFactory>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<MultiClientAuthAttribute>();
builder.Services.AddScoped<StudentService>();
```

API is fully decoupled and DI-friendly for all layers.

20.8 Security Best Practices

1. **Use HTTPS** for all requests → prevents token interception.
2. **HttpOnly cookies** for browsers → prevent XSS attacks.
3. **Bearer tokens** for mobile/embedded clients → secure and portable.
4. **Token expiration and revocation** → prevent misuse of stolen tokens.
5. **Centralized token management in BLL** → ensures separation of concerns.

20.9 Summary

- **Multi-client API authentication** is now supported via **HttpOnly cookies** and **Bearer tokens**.
- **BLL handles all token logic**, ensuring decoupled 3-tier architecture.
- **IAsyncAuthorizationFilter** enforces authentication asynchronously for scalability.
- API endpoints are secure and can be consumed safely by **any client type**.
- Fully compatible with **modern web apps, mobile apps, and embedded devices**.

Your API is now **universal, secure, and maintainable**, with clear separation between **API, BLL, and DAL**.